

复习思维导图

神经网络基本概念	
前向传播	
用构建网络	
(自动求导)	
反向传播 链式法则	
优化器	
训练循环	
推理 预测	
初始化	
正则化	
梯度消失 梯度爆炸	
常见坑 易错点	
与之前学过内容的连接	
代码模板 (来自)	

没有，内容来自份 ()。

神经网络基本概念

从逻辑回归出发

- 单层 (线性 输出)
- 神经网络 堆多层 加非线性激活 表达更复杂的函数

基本结构

- 输入层 : 特征向量 $\mathbf{x} \in \mathbb{R}^d$
- 隐藏层 : 一层或多层, 每层做 $\mathbf{z} = \phi(W\mathbf{x} + \mathbf{b})$
- 输出层
 - 回归: 标量 向量
 - 二分类: 个
 - 多分类: K 个

每层的两步

- 线性变换: $\mathbf{z} = W\mathbf{x} + \mathbf{b}$ (仿射)
- 非线性激活: $\mathbf{z} = \phi(\mathbf{z})$ (关键! 没它整个网络等价于一个线性模型)

- 一个足够宽的单隐层网络 (用合理的激活函数) 可以任意精度逼近任何连续函数

- □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- $$\mathbb{P}(\mathbf{X} = \mathbf{x}) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- □ □ □ □ □ □ □ □ □ □ □ □ □

- $$L = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

- $$L = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log (1 - \hat{p}_i)]$$

- $$L = -\frac{1}{n} \sum_i \log \hat{p}_{i,y_i}$$

- `nn.CrossEntropyLoss` : 内部已含 `nn.LogSoftmax` 和 `nn.NLLLoss` , 输入是 `log-probabilities` (不是概率!)
- 等价方案: `nn.LogSoftmax(dim=1)` □ `nn.NLLLoss`
- 常见错误: `nn.CrossEntropyLoss` 喂 `probabilities` 后的概率 □ 等于做了两次 `nn.LogSoftmax` , `nn.NLLLoss` 偏小且训练慢

□ `nn.Module` 和 `nn.Sequential`

- 类似 `nn.Module` 但有梯度跟踪
- 维度名词
 - `nn.Module` 的 `input_size` 和 `output_size` 是 `nn.Module` 的 `input_size` 和 `output_size` 的 `nn.Module` 的 `input_size` 和 `output_size`
- 创建
 - `torch.tensor([1,2,3])`、`torch.zeros(3,4)`、`torch.ones`、`torch.randn`、`torch.arange`
- 形状操作
 - `view` □ `reshape` (`view` 更安全, `reshape` 要求连续内存)
 - `unsqueeze(dim)`: 加一维 □ `squeeze`: 去掉长度 1 的维
 - `resize_`: 原地改形状 (_ 后缀 □ `resize`)
- 运算
 - 元素级: `+` `-` `*` `/` `**`
 - 矩阵乘: `@`、`torch.matmul`、`torch.mm`
 - 聚合: `.sum()` □ `.mean()` □ `.max()` □ `.argmax()` 等
- 广播 `broadcasting`: 和 `numpy` 同规则
- `tensor.numpy()` □ `torch.from_numpy(arr)`
- 共享内存: 改一个另一个也变
- `tensor.to('cuda')` □ `tensor.to(device)`
- 不同 `device` 间运算会报错
- 默认 `float32`
- `torch.float16` □ `torch.float64` □ `torch.long` 等

□ 前向传播 `forward` 和 `backward`

- 单层手算: $y = \phi(Wx + b)$
- 多层: 上一层 `output` 是 下一层 `input`
- 完整例子: `nn.Module`
 - 输入: `input_size` 维 (`input_size` 像素 `input_size`)
 - 隐藏层 1: `hidden_size1` (`hidden_size1`)
 - 隐藏层 2: `hidden_size2` (`hidden_size2`)
 - 输出层: `output_size` (`output_size` 类)
- 形状检查 (关键习惯)
 - 每层 `W` 形状: (`input_size`, `output_size`)
 - `nn.Linear(in, out)` 的 `weight` 形状是 (`output_size`, `input_size`) (注意倒过来)
 - 输出 □ 输入 $@ W^T$ □ `b`

□ 用 `nn.Module` 构建网络

- 两种写法
 - `nn.Module` 写法 (继承 `nn.Module`): 灵活, 能加自定义逻辑
 - `nn.Sequential` 写法: 层简单堆叠时更短
- `nn.Module` 写法的规矩
 - 必须 `super().__init__()`
 - 在 `__init__` 里定义所有 `nn.Module` (`self.fc1 = nn.Linear(...)`)
 - 在 `forward(x)` 里写前向计算
- `nn.Module` 的 `name` 给层命名: 调试时方便看权重 `nn.Module`
- `F.relu` □ `nn.ReLU()`
 - `F.relu(x)` 是函数式调用


```

zeros = torch.zeros(2, 3)
ones = torch.ones_like(random)

#
random.shape, random.dtype, random.device

#
tensor + 10
torch.matmul(A, B) #
A @ B #

# Numpy
np_arr = tensor.numpy()
tensor = torch.from_numpy(np_arr)

# GPU
device = "cuda" if torch.cuda.is_available() else "cpu"
tensor.to(device)

□ □ □ □ □ 数据加载

from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)),
])

trainset = datasets.MNIST('~/.pytorch/MNIST_data/',
                          download=True, train=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

□ 构建网络 ( □ □ □ □ □ 写法 )

import torch.nn as nn
import torch.nn.functional as F

class Network(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x) # logits
        return x

□ 构建网络 ( □ □ □ □ □ □ □ □ □ □ 写法 )

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
    nn.LogSoftmax(dim=1), # NLLLoss
)

□ □ □ □ □ 配对

```

```

# A raw logits + CrossEntropyLoss
model = nn.Sequential(..., nn.Linear(64, 10)) # softmax
criterion = nn.CrossEntropyLoss()

# B LogSoftmax + NLLLoss
model = nn.Sequential(..., nn.Linear(64, 10), nn.LogSoftmax(dim=1))
criterion = nn.NLLLoss()

□ 完整训练循环

from torch import optim

model = nn.Sequential(
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 64), nn.ReLU(),
    nn.Linear(64, 10), nn.LogSoftmax(dim=1),
)
criterion = nn.NLLLoss()
optimizer = optim.SGD(model.parameters(), lr=0.003)

epochs = 20
for epoch in range(epochs):
    running_loss = 0
    for images, labels in trainloader:
        images = images.view(images.shape[0], -1) # flatten

        optimizer.zero_grad() # 1.
        logits = model(images) # 2. forward
        loss = criterion(logits, labels) # 3. loss
        loss.backward() # 4. backward
        optimizer.step() # 5. update

        running_loss += loss.item()
    print(f"Epoch {epoch}, loss: {running_loss / len(trainloader)}")

□ 推理 □ 预测

model.eval()
with torch.no_grad():
    logits = model(x_test)
    probs = torch.softmax(logits, dim=1)
    pred = probs.argmax(dim=1)

□ □ □ □ □ □ □ 演示

#
loss.backward()
print(model[0].weight.grad) #

#
with torch.no_grad():
    preds = model(x_test)

□ □ □ □ 优化器

optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

□ 学习率调度

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
for epoch in range(epochs):
    train(...)
    scheduler.step()

```